In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
df=pd.read_csv('Churn_Modelling.csv')
```

In [3]:
```
df.head()
```

Out[3]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	B
0	1	15634602	Hargrave	619	France	Female	42	2	
1	2	15647311	Hill	608	Spain	Female	41	1	83
2	3	15619304	Onio	502	France	Female	42	8	159
3	4	15701354	Boni	699	France	Female	39	1	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

In [5]:
```
df.isnull().sum()
```

Out[5]:

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0

dtype: int64

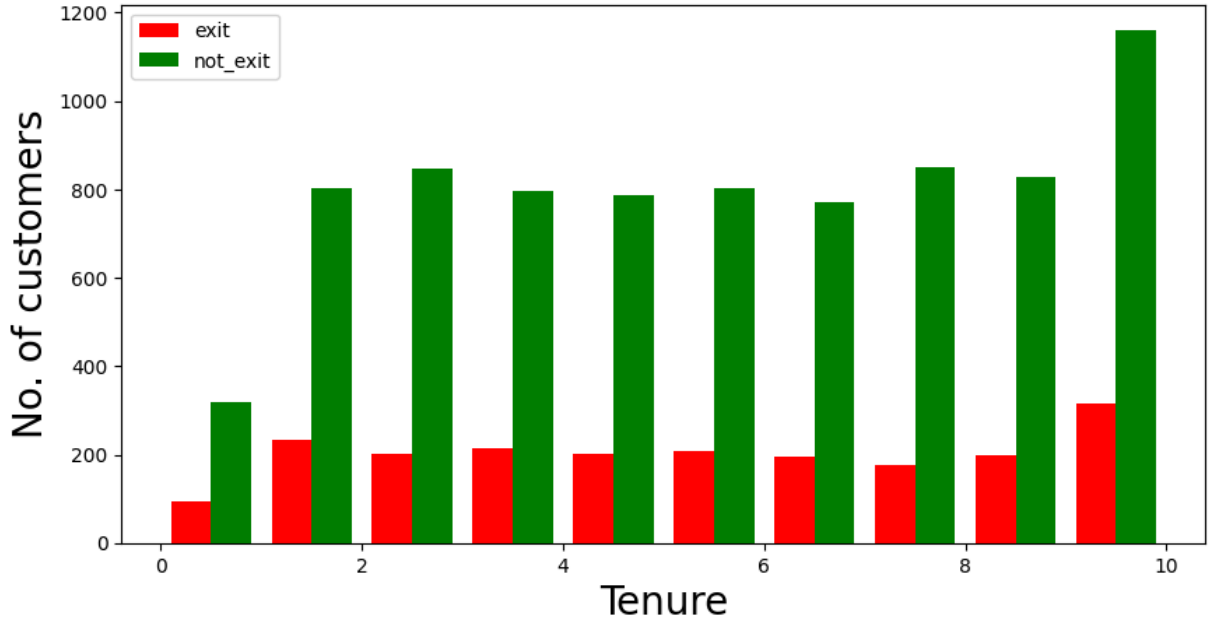
In [6]:
```
df.columns
```

Out[6]:
```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

In [7]:
```
def visualization(x, y, xlabel):
    plt.figure(figsize=(10,5))
    plt.hist([x, y], color=['red', 'green'], label = ['exit', 'not_exit'])
    plt.xlabel(xlabel,fontsize=20)
    plt.ylabel("No. of customers", fontsize=20)
    plt.legend()
```

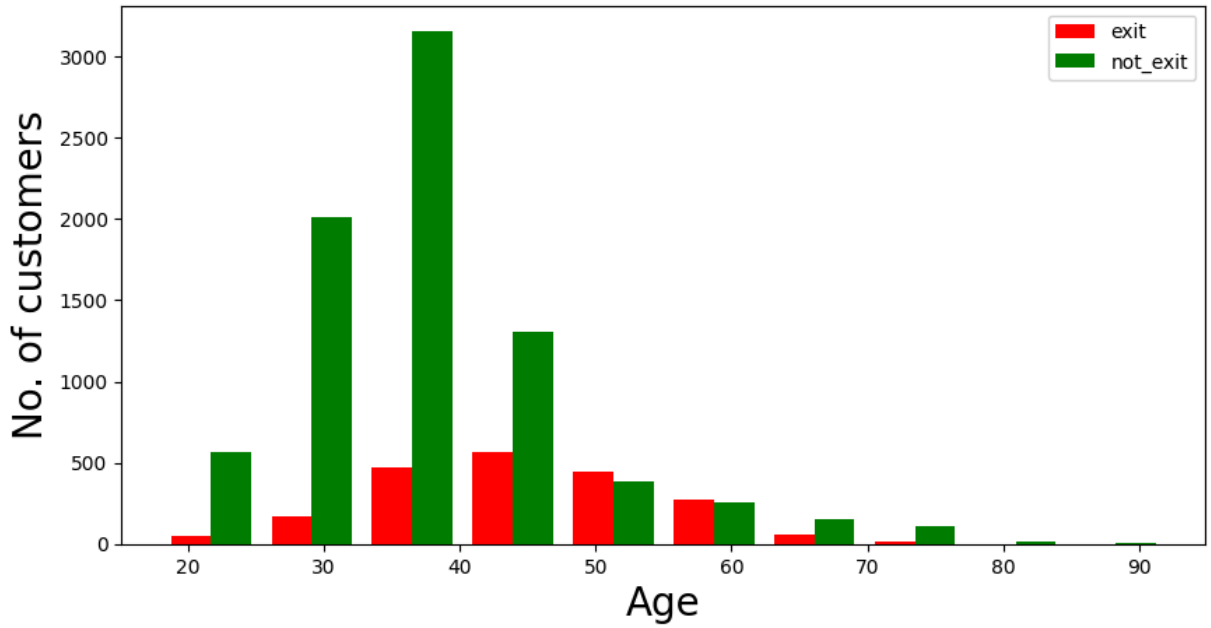
In [8]:
```
df_churn_exited = df[df['Exited']==1]['Tenure']
df_churn_not_exited = df[df['Exited']==0]['Tenure']
```

In [9]:
```
visualization(df_churn_exited, df_churn_not_exited, "Tenure")
```



In [10]:
```
df_churn_exited2 = df[df['Exited']==1]['Age']
df_churn_not_exited2 = df[df['Exited']==0]['Age']
```

In [11]:
```
visualization(df_churn_exited2,df_churn_not_exited2, 'Age')
```



In [12]:
```
X = df[['CreditScore','Gender','Age','Tenure','Balance','NumOfProducts','HasCrCard'
states = pd.get_dummies(df['Geography'],drop_first = True)
gender = pd.get_dummies(df['Gender'],drop_first = True)
```

In [13]:
```
gender
```

Out[13]:

	Male
0	False
1	False
2	False
3	False
4	False
...	...
9995	True
9996	True
9997	False
9998	True
9999	False

10000 rows × 1 columns

In [14]:
```
df = pd.concat([df,gender,states], axis = 1)
df
```

Out[14]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
0	1	15634602	Hargrave	619	France	Female	42	2
1	2	15647311	Hill	608	Spain	Female	41	1
2	3	15619304	Onio	502	France	Female	42	8
3	4	15701354	Boni	699	France	Female	39	1
4	5	15737888	Mitchell	850	Spain	Female	43	2
...	...	...	...	...	...	...	...	...
9995	9996	15606229	Obijiaku	771	France	Male	39	5
9996	9997	15569892	Johnstone	516	France	Male	35	10
9997	9998	15584532	Liu	709	France	Female	36	7
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3
9999	10000	15628319	Walker	792	France	Female	28	4

10000 rows × 17 columns

In [15]:
```
X = df[['CreditScore','Age','Tenure','Balance','NumOfProducts','HasCrCard','IsActiv
y=df[['Exited']]
```

In [16]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.30)
```

In [17]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

In [18]:
```
X_train  = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [19]:
```
import keras
```

In [21]:
```
import keras
```

In [22]:
```
from keras.models import Sequential #To create sequential neural network
from keras.layers import Dense #To create hidden layers
```

In [23]:
```
classifier = Sequential()
```

In [24]:
```
classifier.add(Dense(activation = "relu",input_dim = 11,units = 6,kernel_initialize
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarni
ng: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequent
ial models, prefer using an `Input(shape)` object as the first layer in the model in
stead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [25]:
```
classifier.add(Dense(activation = "relu",units = 6,kernel_initializer = "uniform"))
```

In [26]:
```
classifier.add(Dense(activation = "sigmoid",units = 1,kernel_initializer = "uniform
```

In [27]:
```
classifier.compile(optimizer="adam",loss = 'binary_crossentropy',metrics = ['accura
```

In [28]:
```
classifier.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6)	72
dense_1 (Dense)	(None, 6)	42
dense_2 (Dense)	(None, 1)	7

Total params: 121 (484.00 B)

Trainable params: 121 (484.00 B)

Non-trainable params: 0 (0.00 B)

In [29]:
```
classifier.fit(X_train,y_train,batch_size=10,epochs=50)
```

```
Epoch 1/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.7885 - loss: 0.5724
Epoch 2/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.7931 - loss: 0.4311
Epoch 3/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8104 - loss: 0.4132
Epoch 4/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8322 - loss: 0.4095
Epoch 5/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8282 - loss: 0.4154
Epoch 6/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.8376 - loss: 0.4011
Epoch 7/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8374 - loss: 0.4032
Epoch 8/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8368 - loss: 0.4138
Epoch 9/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8361 - loss: 0.4032
Epoch 10/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8423 - loss: 0.3985
Epoch 11/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8312 - loss: 0.4039
Epoch 12/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8459 - loss: 0.3820
Epoch 13/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8375 - loss: 0.3995
Epoch 14/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8412 - loss: 0.3903
Epoch 15/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8450 - loss: 0.3819
Epoch 16/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8466 - loss: 0.3848
Epoch 17/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8421 - loss: 0.3913
Epoch 18/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8486 - loss: 0.3798
Epoch 19/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8388 - loss: 0.3948
Epoch 20/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8438 - loss: 0.3897
Epoch 21/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8407 - loss: 0.3928
Epoch 22/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8495 - loss: 0.3753
Epoch 23/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8427 - loss: 0.3785
Epoch 24/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8378 - loss: 0.3939
Epoch 25/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8445 - loss: 0.3850
Epoch 26/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8399 - loss: 0.3945
Epoch 27/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8410 - loss: 0.3909
Epoch 28/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8454 - loss: 0.3832
```

```
Epoch 29/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8431 - loss: 0.3842
Epoch 30/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8413 - loss: 0.3863
Epoch 31/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8411 - loss: 0.3939
Epoch 32/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8398 - loss: 0.3983
Epoch 33/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8389 - loss: 0.3984
Epoch 34/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8420 - loss: 0.3930
Epoch 35/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8476 - loss: 0.3813
Epoch 36/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8435 - loss: 0.3842
Epoch 37/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8445 - loss: 0.3864
Epoch 38/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8493 - loss: 0.3777
Epoch 39/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8405 - loss: 0.3911
Epoch 40/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8532 - loss: 0.3728
Epoch 41/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8483 - loss: 0.3762
Epoch 42/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8414 - loss: 0.3893
Epoch 43/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8396 - loss: 0.3943
Epoch 44/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8427 - loss: 0.3887
Epoch 45/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8486 - loss: 0.3751
Epoch 46/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8512 - loss: 0.3721
Epoch 47/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8477 - loss: 0.3828
Epoch 48/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8515 - loss: 0.3804
Epoch 49/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8416 - loss: 0.3904
Epoch 50/50
700/700 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8470 - loss: 0.3875
```

Out[29]: <keras.src.callbacks.history.History at 0x7c50e8180320>

In [30]:
```
y_pred =classifier.predict(X_test)
print(y_pred)
print(y_pred.shape)
y_pred = (y_pred > 0.5)
print(y_pred)
```

```
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
[[0.17612754]
 [0.2760512 ]
 [0.11314794]
 ...
 [0.17684211]
 [0.0287735 ]
 [0.0354096 ]]
(3000, 1)
[[False]
 [False]
 [False]
 ...
 [False]
 [False]
 [False]]
```

In [31]:
```
from sklearn.metrics import confusion_matrix,accuracy_score,classification_report
```

In [32]:
```
cm = confusion_matrix(y_test,y_pred)
```

In [33]:
```
cm
```

Out[33]:
```
array([[2312,   58],
       [ 464,  166]])
```

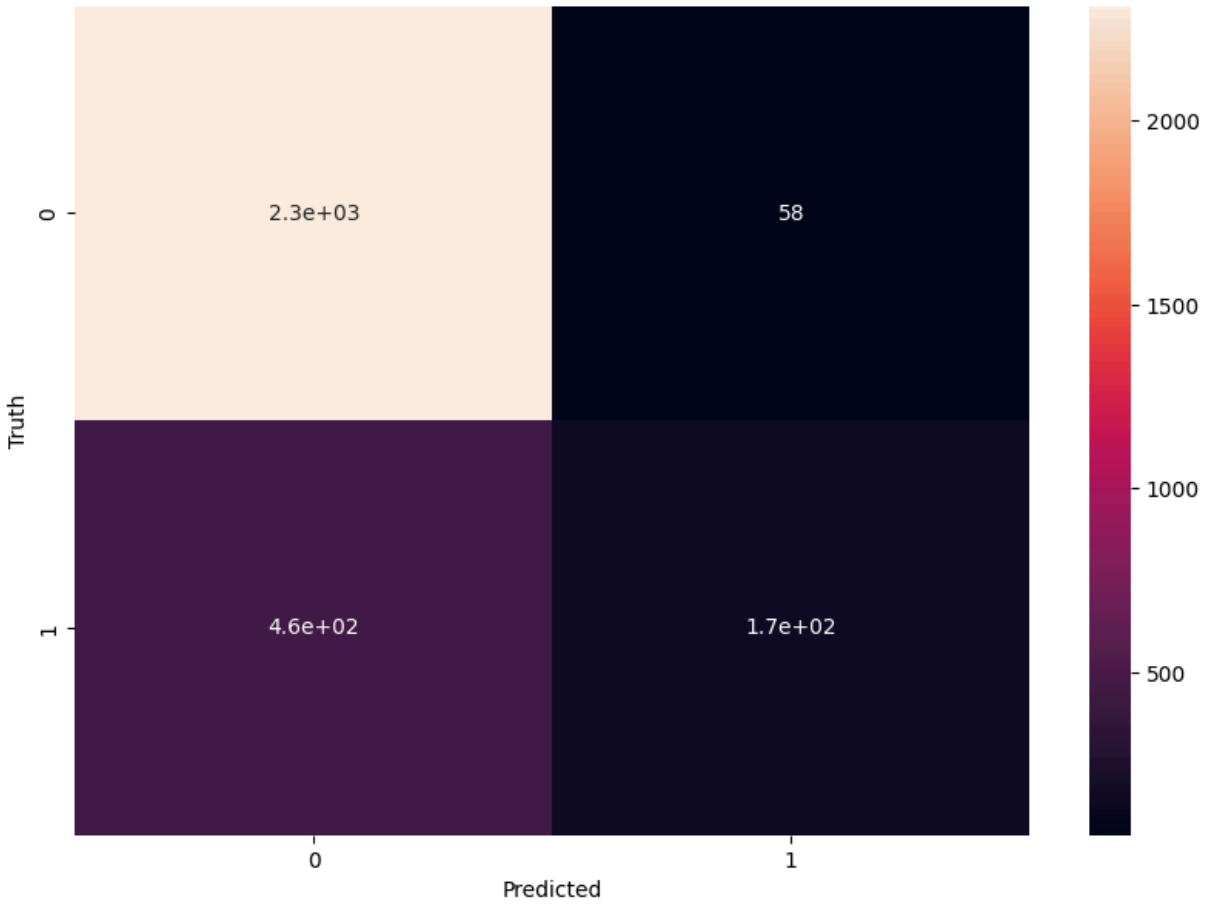
In [34]:
```
accuracy = accuracy_score(y_test,y_pred)
```

In [35]:
```
accuracy
```

Out[35]:
```
0.826
```

In [36]:
```
plt.figure(figsize = (10,7))
sns.heatmap(cm,annot = True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

Out[36]:
```
Text(95.72222222222221, 0.5, 'Truth')
```



In [37]:
```
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.98      0.90      2370
           1       0.74      0.26      0.39       630

    accuracy                           0.83      3000
   macro avg       0.79      0.62      0.64      3000
weighted avg       0.81      0.83      0.79      3000
```

In [38]:
```
new_customer = [[0, 0, 600, 1, 40, 3, 60000, 2, 1, 1, 50000]]
```

In [39]:
```
new_customer = sc.transform(sc.transform(new_customer))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarnin
g: X does not have valid feature names, but StandardScaler was fitted with feature n
ames
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarnin
g: X does not have valid feature names, but StandardScaler was fitted with feature n
ames
  warnings.warn(
```

In [40]:
```
new_prediction = classifier.predict(new_customer)
new_prediction
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
```

Out[40]:
```
array([[0.]], dtype=float32)
```

In [ ]:
```
new_prediction = (new_prediction > 0.5)
new_prediction
```

Out[ ]:
```
array([[False]])
```

In [ ]:
```

```